

Cơ chế Attention và kiến trúc Transformer

Bài giảng môn Deep Learning

Khoa học Dữ liệu

Giảng viên: Hà Minh Tuấn

Năm học 2025 – 2026

Abstract

Tài liệu này trình bày toàn diện về **cơ chế Attention** và **kiến trúc Transformer** — nền tảng của các mô hình ngôn ngữ lớn hiện đại như BERT, GPT, và Vision Transformer (ViT). Nội dung bao gồm: lý thuyết chi tiết kèm công thức toán học, ví dụ minh họa trực quan, sơ đồ kiến trúc, và mã nguồn PyTorch hoàn chỉnh. Tài liệu phù hợp cho sinh viên đã có kiến thức cơ bản về mạng nơ-ron (neural networks) và xử lý ngôn ngữ tự nhiên (NLP).

Contents

1	Giới thiệu về cơ chế Attention	3
1.1	Attention là gì?	3
1.1.1	Ý tưởng cốt lõi — Giải thích từng bước	3
1.2	Các loại Attention: Bahdanau và Luong	7
1.2.1	Bahdanau Attention (Additive Attention, 2014)	8
1.2.2	Luong Attention (Multiplicative Attention, 2015)	8
2	Self-Attention (Tự chú ý)	8
2.1	Khái niệm và động lực	8
2.2	Scaled Dot-Product Attention	9
2.2.1	Ba thành phần Q, K, V	9
2.2.2	Công thức Scaled Dot-Product Attention	11
2.2.3	Ví dụ tính toán cụ thể	12
3	Multi-Head Attention (Attention đa đầu)	13
3.1	Mục đích và kiến trúc	13
3.2	Chi tiết triển khai	14
4	Kiến trúc Transformer	14
4.1	Cấu trúc Encoder-Decoder	14
4.2	Positional Encoding (Mã hoá vị trí)	14
4.3	Feed-Forward Networks (FFN)	18

4.4	Layer Normalization và Residual Connections	18
4.4.1	Residual Connection (Kết nối tắt)	18
4.4.2	Layer Normalization	18
4.5	Ví dụ tổng hợp: Transformer dịch một câu từ đầu đến cuối	19
4.5.1	Bước 1: Input Embedding — Chuyển từ thành vector	19
4.5.2	Bước 2: Cộng Positional Encoding	20
4.5.3	Bước 3: Encoder Self-Attention — “Mỗi từ nhìn tất cả từ khác”	20
4.5.4	Bước 3.5: Add & Norm — Residual + Layer Normalization	22
4.5.5	Bước 4: Encoder FFN — Biến đổi phi tuyến	22
4.5.6	Bước 5: Output Embedding + Positional Encoding (Decoder Input)	23
4.5.7	Bước 6: Decoder Masked Self-Attention — “Không được nhìn tương lai”	24
4.5.8	Bước 7: Cross-Attention — “Decoder hỏi Encoder”	24
4.5.9	Bước 8: Decoder FFN + Linear + Softmax — Sinh từ tiếp theo	26
4.5.10	Tổng kết: Toàn bộ quá trình dịch	27
5	Xây dựng Transformer Block	27
5.1	Encoder Block	27
5.2	Decoder Block	28
6	Ứng dụng của Transformer	29
6.1	Xử lý ngôn ngữ tự nhiên (NLP)	29
6.2	Vision Transformers (ViT)	30
7	Triển khai Transformer đơn giản với PyTorch	30
7.1	Positional Encoding	30
7.2	Scaled Dot-Product Attention (Tự triển khai)	31
7.3	Multi-Head Attention (Tự triển khai)	31
7.4	Transformer Model hoàn chỉnh	32
8	Các mô hình Transformer huấn luyện sẵn	33
8.1	Tổng quan các mô hình phổ biến	33
8.1.1	BERT (Bidirectional Encoder Representations from Transformers)	33
8.1.2	GPT (Generative Pre-trained Transformer)	33
8.2	Sử dụng thư viện Hugging Face Transformers	34
9	Fine-tuning mô hình Transformer huấn luyện sẵn	35
9.1	Khái niệm và kỹ thuật	35
9.2	Ví dụ: Phân loại văn bản với Hugging Face	36
9.3	Ví dụ nhanh: Fine-tuning với LoRA	37
	Bảng tóm tắt	38
	Tài liệu tham khảo	38

1 Giới thiệu về cơ chế Attention

1.1 Attention là gì?

Trong các mô hình Sequence-to-Sequence (Seq2Seq) truyền thống, bộ mã hoá (encoder) nén toàn bộ câu đầu vào thành *một vector ngữ cảnh duy nhất* (context vector). Điều này gây ra hiện tượng **nghẽn cổ chai thông tin** (information bottleneck), đặc biệt với các câu dài.

Định nghĩa: Attention

Cơ chế Attention cho phép mô hình tại mỗi bước giải mã (decoding step) *nhìn lại* tất cả các trạng thái ẩn của bộ mã hoá và chọn lọc thông tin quan trọng nhất, thay vì chỉ dựa vào một vector ngữ cảnh cố định.

Ví dụ: Dịch máy

Xét bài toán dịch câu: “Con mèo ngồi trên thảm” → “The cat sits on the mat”.

Khi giải mã từ “cat”, mô hình cần **tập trung** (attend) vào từ “mèo” trong câu nguồn chứ không phải “thảm”. Cơ chế Attention tự động học được sự tập trung này.

1.1.1 Ý tưởng cốt lõi — Giải thích từng bước

Để hiểu rõ cơ chế Attention, ta cần hiểu mô hình Seq2Seq gồm hai phần chạy **lần lượt theo thời gian**:

Ghi chú: Nhắc lại: Encoder và Decoder là gì?

Encoder (bộ mã hoá) là một mạng RNN đọc câu nguồn từ trái sang phải. Tại mỗi từ thứ i , nó tạo ra một **trạng thái ẩn** (hidden state) h_i — đây là một vector chứa “hiểu biết” của mô hình sau khi đã đọc đến từ thứ i .

Decoder (bộ giải mã) là một mạng RNN khác, sinh ra câu đích từng từ một. Tại mỗi bước t , nó có **trạng thái ẩn** s_t — chứa “hiểu biết” của decoder sau khi đã sinh ra t từ.

Bước 0: Encoder tạo các trạng thái ẩn. Encoder đọc câu nguồn “Con mèo ngồi trên” (4 từ, $T_x = 4$) và tạo ra 4 trạng thái ẩn:

$$h_1 (\text{“Con”}), \quad h_2 (\text{“mèo”}), \quad h_3 (\text{“ngồi”}), \quad h_4 (\text{“trên”})$$

Mỗi h_i là một vector (ví dụ 256 chiều) tóm tắt thông tin từ đầu câu đến từ thứ i .

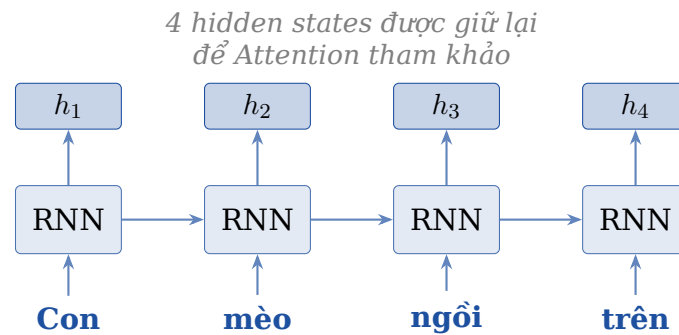


Figure 1: Encoder RNN đọc câu nguồn và tạo ra các trạng thái ẩn h_1, h_2, h_3, h_4 .

Bước 1: Decoder bắt đầu giải mã — s_{t-1} là gì? Decoder sinh từng từ theo thứ tự. s_{t-1} là trạng thái ẩn của decoder ở bước trước đó — nó chứa “ngữ cảnh decoder đã tích lũy” tính đến thời điểm trước bước hiện tại.

Ví dụ: Minh họa cụ thể các bước decoder

Giả sử ta đang dịch sang tiếng Anh: “The cat sits on”:

Bước t	s_{t-1}	Từ đang sinh	Ý nghĩa s_{t-1}
$t = 1$	s_0 (khởi tạo)	“The”	Chưa biết gì, trạng thái ban đầu
$t = 2$	s_1 (sau “The”)	“cat”	Đã biết từ trước là “The”
$t = 3$	s_2 (sau “The cat”)	“sits”	Đã biết “The cat”
$t = 4$	s_3 (sau “The cat sits”)	“on”	Đã biết “The cat sits”

Khi sinh từ “cat” ($t = 2$), decoder dùng s_1 (trạng thái sau khi đã sinh “The”) để **hỏi** encoder: “Ở bước này, tôi nên tập trung vào từ nào trong câu nguồn?”

Bước 2: Tính điểm tương thích $e_{t,i}$ — “Từ nào quan trọng?” $e_{t,i}$ đo mức độ **liên quan** giữa những gì decoder đang cần (s_{t-1}) và thông tin tại vị trí i của encoder (h_i). Có thể hiểu $e_{t,i}$ như câu hỏi: “Khi tôi đang ở trạng thái s_{t-1} , thì trạng thái encoder h_i quan trọng đến mức nào?”

Cách tính $e_{t,i}$ phụ thuộc vào loại attention. Hai cách phổ biến nhất:

Cách 1: Additive (Bahdanau) — dùng mạng nơ-ron nhỏ

$$e_{t,i} = v^T \tanh(W_1 s_{t-1} + W_2 h_i) \quad (1)$$

Ý tưởng: Đặt s_{t-1} và h_i vào một mạng nơ-ron 1 lớp (có tanh), rồi dùng vector v để “chấm điểm”. Các tham số W_1, W_2, v được học trong quá trình huấn luyện.

Cách 2: Dot-product (Luong) — đơn giản hơn

$$e_{t,i} = s_t^T h_i \quad (\text{tích vô hướng}) \quad (2)$$

Ý tưởng: Nếu hai vector s_t và h_i “chỉ cùng hướng” (nội dung tương tự), tích vô hướng sẽ lớn $\Rightarrow$ điểm cao.

Ví dụ: Tính $e_{t,i}$ bằng số cụ thể (Dot-product)

Giả sử tại bước $t = 2$ (đang sinh từ “cat”), decoder có trạng thái:

$$s_1 = \begin{pmatrix} 0.8 \\ 0.1 \\ 0.5 \end{pmatrix}$$

Và encoder có 4 trạng thái ẩn (mỗi cái 3 chiều cho đơn giản):

$$h_1 = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \end{pmatrix}, h_2 = \begin{pmatrix} 0.9 \\ 0.1 \\ 0.4 \end{pmatrix}, h_3 = \begin{pmatrix} 0.2 \\ 0.7 \\ 0.1 \end{pmatrix}, h_4 = \begin{pmatrix} 0.3 \\ 0.1 \\ 0.6 \end{pmatrix}$$

Tính tích vô hướng $e_{2,i} = s_1^T h_i$:

$$e_{2,1} = 0.8 \times 0.1 + 0.1 \times 0.2 + 0.5 \times 0.3 = 0.08 + 0.02 + 0.15 = \mathbf{0.25}$$

$$e_{2,2} = 0.8 \times 0.9 + 0.1 \times 0.1 + 0.5 \times 0.4 = 0.72 + 0.01 + 0.20 = \mathbf{0.93}$$

$$e_{2,3} = 0.8 \times 0.2 + 0.1 \times 0.7 + 0.5 \times 0.1 = 0.16 + 0.07 + 0.05 = \mathbf{0.28}$$

$$e_{2,4} = 0.8 \times 0.3 + 0.1 \times 0.1 + 0.5 \times 0.6 = 0.24 + 0.01 + 0.30 = \mathbf{0.55}$$

$\Rightarrow e_{2,2} = 0.93$ **là cao nhất** — nghĩa là h_2 (từ “mèo”) liên quan nhất đến decoder lúc này!

Bước 3: Chuyển điểm $e_{t,i}$ thành trọng số $\alpha_{t,i}$ qua Softmax. Các điểm $e_{t,i}$ có thể là bất kỳ số thực nào (âm, dương, lớn, nhỏ). Ta cần chuyển chúng thành **xác suất** (tổng bằng 1, mỗi giá trị trong $[0, 1]$) bằng hàm **softmax**:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T_x} \exp(e_{t,j})} \quad (3)$$

Ví dụ: Tiếp tục ví dụ trên — Tính α bằng softmax

Từ $e_{2,1} = 0.25$, $e_{2,2} = 0.93$, $e_{2,3} = 0.28$, $e_{2,4} = 0.55$:

$$\begin{aligned}\exp(e_{2,1}) &= e^{0.25} = 1.284, & \exp(e_{2,2}) &= e^{0.93} = 2.535 \\ \exp(e_{2,3}) &= e^{0.28} = 1.323, & \exp(e_{2,4}) &= e^{0.55} = 1.733\end{aligned}$$

Tổng: $1.284 + 2.535 + 1.323 + 1.733 = 6.875$

$$\begin{aligned}\alpha_{2,1} &= 1.284/6.875 = \mathbf{0.187} & (\text{"Con"} - \text{ít quan trọng}) \\ \alpha_{2,2} &= 2.535/6.875 = \mathbf{0.369} & (\text{"mèo"} - \mathbf{\text{quan trọng nhất!}}) \\ \alpha_{2,3} &= 1.323/6.875 = \mathbf{0.192} & (\text{"ngồi"} - \text{ít quan trọng}) \\ \alpha_{2,4} &= 1.733/6.875 = \mathbf{0.252} & (\text{"trên"} - \text{trung bình})\end{aligned}$$

Kiểm tra: $0.187 + 0.369 + 0.192 + 0.252 = 1.000 \checkmark$

$\Rightarrow$ Mô hình “chú ý” nhiều nhất vào từ “mèo” ($\alpha = 0.369$) khi đang sinh từ “cat”.

Bước 4: Tính vector ngữ cảnh c_t — “Tổng hợp thông tin”. Vector ngữ cảnh là **tổ hợp tuyến tính có trọng số** của tất cả trạng thái encoder:

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i = \alpha_{t,1} \cdot h_1 + \alpha_{t,2} \cdot h_2 + \alpha_{t,3} \cdot h_3 + \alpha_{t,4} \cdot h_4 \quad (4)$$

Hiểu đơn giản: c_t là “bản tóm tắt có chọn lọc” của câu nguồn, trong đó những từ quan trọng (trọng số α cao) đóng góp nhiều hơn.

Ví dụ: Tiếp tục — Tính c_2

$$c_2 = 0.187 \cdot h_1 + \mathbf{0.369} \cdot h_2 + 0.192 \cdot h_3 + 0.252 \cdot h_4$$

Thay số (với h_i 3 chiều ở trên):

$$\begin{aligned}c_2 &= 0.187 \begin{pmatrix} 0.1 \\ 0.2 \\ 0.3 \end{pmatrix} + 0.369 \begin{pmatrix} 0.9 \\ 0.1 \\ 0.4 \end{pmatrix} + 0.192 \begin{pmatrix} 0.2 \\ 0.7 \\ 0.1 \end{pmatrix} + 0.252 \begin{pmatrix} 0.3 \\ 0.1 \\ 0.6 \end{pmatrix} \\ &= \begin{pmatrix} 0.019 \\ 0.037 \\ 0.056 \end{pmatrix} + \begin{pmatrix} 0.332 \\ 0.037 \\ 0.148 \end{pmatrix} + \begin{pmatrix} 0.038 \\ 0.134 \\ 0.019 \end{pmatrix} + \begin{pmatrix} 0.076 \\ 0.025 \\ 0.151 \end{pmatrix} = \begin{pmatrix} 0.465 \\ 0.233 \\ 0.374 \end{pmatrix}\end{aligned}$$

Vector c_2 này mang nặng thông tin từ h_2 (“mèo”) nhất, và được đưa vào decoder để giúp sinh ra từ “cat”.

Bước 5: Decoder dùng c_t để sinh từ tiếp theo. Decoder kết hợp c_t với trạng thái hiện tại để dự đoán từ:

$$s_t = f(s_{t-1}, y_{t-1}, c_t) \quad \text{và} \quad P(y_t | y_{<t}, X) = g(s_t, c_t) \quad (5)$$

trong đó y_{t-1} là từ vừa sinh ở bước trước, f là hàm RNN (GRU/LSTM), và g thường là một lớp tuyến tính + softmax trên toàn bộ từ vựng.

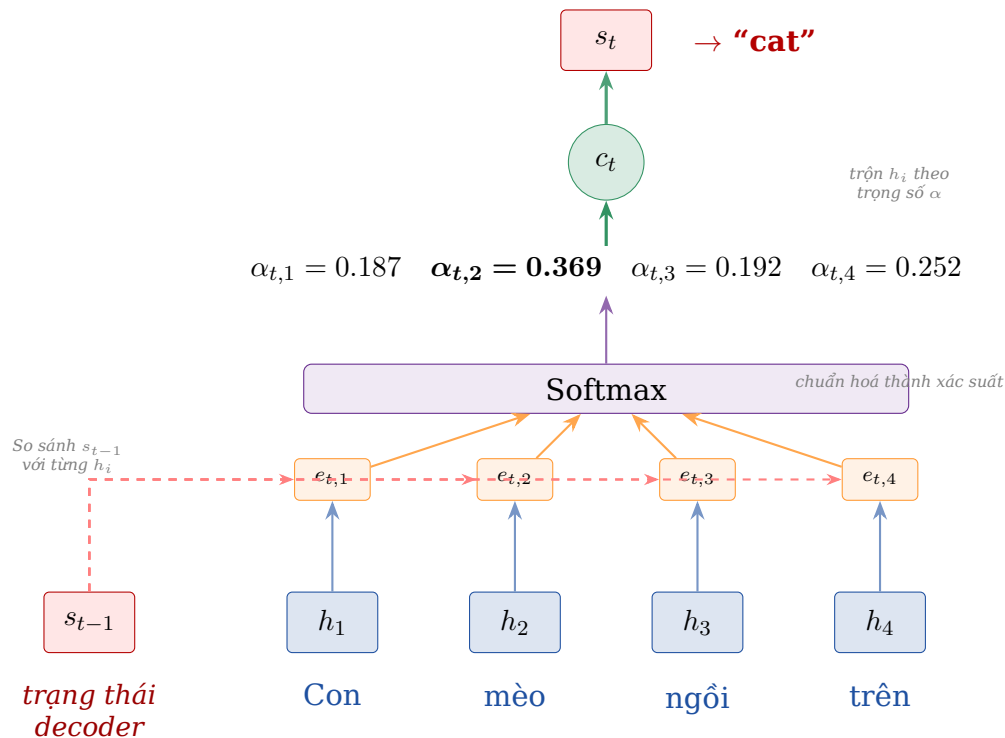


Figure 2: Toàn bộ pipeline Attention: s_{t-1} (decoder) so sánh với từng h_i (encoder) $\rightarrow$ tính điểm $e_{t,i}$ $\rightarrow$ softmax thành $\alpha_{t,i}$ $\rightarrow$ trộn h_i thành c_t $\rightarrow$ decoder dùng c_t để sinh từ "cat".

Mẹo: Tóm tắt trực giác

Hãy tưởng tượng bạn đang dịch một câu tiếng Việt sang tiếng Anh. Mỗi khi bạn viết một từ tiếng Anh, bạn *liếc mắt* nhìn lại câu tiếng Việt để xem từ nào liên quan nhất. Attention chính là cơ chế "liếc mắt" đó — nó tự động học được nên nhìn vào đâu.

- s_{t-1} : "Tôi đang ở đâu trong quá trình dịch?" (ngữ cảnh decoder)
- h_i : "Mỗi từ nguồn chứa thông tin gì?" (ngữ cảnh encoder)
- $e_{t,i}$: "Từ nguồn thứ i quan trọng bao nhiêu cho bước dịch hiện tại?" (điểm liên quan)
- $\alpha_{t,i}$: Phiên bản chuẩn hoá của $e_{t,i}$, tổng bằng 1 (xác suất)
- c_t : "Bản tóm tắt có chọn lọc" của câu nguồn cho bước dịch hiện tại

1.2 Các loại Attention: Bahdanau và Luong

Hai biến thể Attention phổ biến nhất trong mô hình Seq2Seq RNN:

1.2.1 Bahdanau Attention (Additive Attention, 2014)

Điểm tương thích được tính qua một mạng nơ-ron feedforward:

$$e_{t,i} = v^T \tanh(W_1 s_{t-1} + W_2 h_i) \quad (6)$$

trong đó W_1, W_2 là ma trận trọng số và v là vector trọng số, tất cả đều được học.

1.2.2 Luong Attention (Multiplicative Attention, 2015)

Đơn giản hơn, dùng tích vô hướng:

$$e_{t,i} = s_t^T h_i \quad (\text{dot}) \quad (7)$$

$$e_{t,i} = s_t^T W_a h_i \quad (\text{general}) \quad (8)$$

$$e_{t,i} = v_a^T \tanh(W_a [s_t; h_i]) \quad (\text{concat}) \quad (9)$$

Table 1: So sánh Bahdanau vs. Luong Attention

Tiêu chí	Bahdanau	Luong
Năm công bố	2014	2015
Loại score	Cộng (additive)	Nhân (multiplicative)
Trạng thái decoder dùng	s_{t-1} (bước trước)	s_t (bước hiện tại)
Độ phức tạp	Cao hơn	Thấp hơn

2 Self-Attention (Tự chú ý)

2.1 Khái niệm và động lực

Trong các cơ chế attention truyền thống ở Mục 1, attention được tính *giữa* hai chuỗi khác nhau (encoder $\leftrightarrow$ decoder). **Self-Attention** cho phép mỗi phần tử trong *cùng một chuỗi* chú ý đến *tất cả các phần tử khác* trong chính chuỗi đó.

Định nghĩa: Self-Attention

Cho chuỗi đầu vào $X = (x_1, x_2, \dots, x_n)$, self-attention tính biểu diễn mới cho mỗi x_i bằng cách kết hợp thông tin từ *tất cả* các vị trí x_j trong chuỗi, với trọng số phụ thuộc vào mức độ liên quan giữa x_i và x_j .

Ví dụ: Hiểu ngữ cảnh

Xét câu: “Con chó đuổi con mèo vì nó chạy nhanh.” Từ “nó” có thể chỉ “chó” hoặc “mèo”. Self-attention giúp mô hình học được mối quan hệ giữa “nó” và các từ khác để xác định đúng nghĩa.

2.2 Scaled Dot-Product Attention

Đây là công thức nền tảng được dùng trong Transformer (Vaswani et al., 2017).

2.2.1 Ba thành phần Q, K, V

Trước khi đi vào công thức, ta cần hiểu rõ các ký hiệu kích thước:

Định nghĩa: d_{model} , d_k , d_v — Ba con số quan trọng nhất

d_{model} — Kích thước biểu diễn của mỗi token (từ) trong mô hình.

Mỗi từ trong câu được biểu diễn bằng một vector có d_{model} chiều. Đây là “ngôn ngữ nội bộ” mà Transformer dùng xuyên suốt — mọi lớp đều nhận vào và trả ra vector d_{model} chiều.

d_k — Kích thước của vector Query và Key (dùng để “so khớp”).

Query (câu hỏi) và Key (chìa khoá) được chiếu xuống không gian d_k chiều trước khi tính điểm tương đồng. Trong Transformer gốc, $d_k = d_{model}/h$, trong đó h là số lượng head.

d_v — Kích thước của vector Value (thông tin thực sự được truyền đi).

Sau khi tính xong trọng số attention, ta dùng chúng để trộn các vector Value. Thường $d_v = d_k$.

Ví dụ: Minh hoạ bằng số cụ thể

Trong Transformer gốc (bài báo “Attention Is All You Need”):

$$d_{model} = 512, \quad h = 8 \text{ heads}, \quad d_k = d_v = \frac{512}{8} = 64$$

Nghĩa là gì?

- Mỗi từ trong câu được biểu diễn bằng 1 vector 512 chiều.
 - Khi tính attention, mỗi head chiếu vector 512 chiều xuống còn 64 chiều để so khớp.
 - 8 head hoạt động song song, mỗi head “nhìn” từ 1 góc khác nhau (64 chiều/head).
 - Sau khi nối (concat) 8 head: $8 \times 64 = 512$ chiều, quay về đúng d_{model} .
- Với mô hình nhỏ hơn (dùng để học/thí nghiệm):

$$d_{model} = 256, \quad h = 4 \text{ heads}, \quad d_k = d_v = \frac{256}{4} = 64$$

Ghi chú: Tại sao không dùng thẳng d_{model} ?

Nếu dùng $d_k = d_{model} = 512$ cho mỗi head, thì:

- Tính QK^T sẽ là phép nhân ma trận $(n \times 512) \times (512 \times n)$ — **rất tốn bộ nhớ và thời gian.**
- 8 head $\times$ 512 chiều = 4096 chiều sau khi concat — **bùng nổ kích thước.**

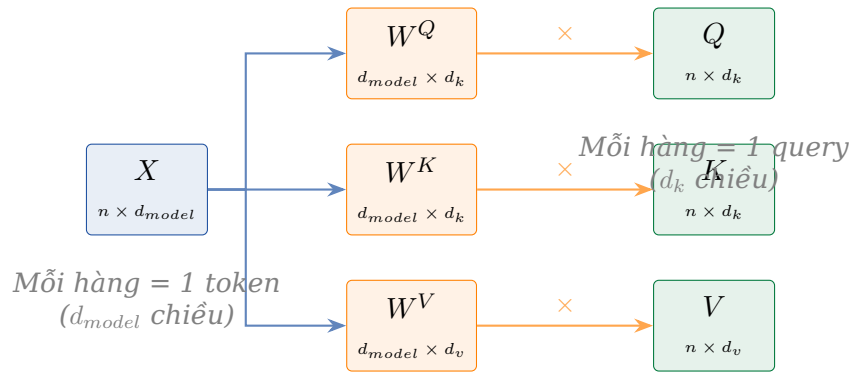
Bằng cách chia $d_k = d_{model}/h$, tổng chi phí tính toán của h head xấp xỉ bằng 1 head duy nhất với $d_k = d_{model}$, nhưng mô hình lại học được **những loại quan hệ** khác nhau.

Bây giờ ta đi vào công thức. Cho ma trận đầu vào $X \in \mathbb{R}^{n \times d_{model}}$ (gồm n token, mỗi token là vector d_{model} chiều), ta chiếu thành ba ma trận:

$$Q = XW^Q, \quad W^Q \in \mathbb{R}^{d_{model} \times d_k} \quad (\text{Query — “Tôi đang tìm gì?”}) \quad (10)$$

$$K = XW^K, \quad W^K \in \mathbb{R}^{d_{model} \times d_k} \quad (\text{Key — “Tôi chứa thông tin gì?”}) \quad (11)$$

$$V = XW^V, \quad W^V \in \mathbb{R}^{d_{model} \times d_v} \quad (\text{Value — “Đây là thông tin của tôi”}) \quad (12)$$



Ví dụ: $n = 10$ từ, $d_{model} = 512$, $d_k = d_v = 64$
 $X: 10 \times 512 \rightarrow Q: 10 \times 64, K: 10 \times 64, V: 10 \times 64$

Figure 3: Chiếu X thành Q, K, V : ma trận X (kích thước $n \times d_{model}$) được nhân với 3 ma trận trọng số khác nhau để tạo ra Query, Key, Value với kích thước nhỏ hơn (d_k chiều).

Ví dụ: Ví dụ cụ thể về kích thước ma trận

Câu “Con mèo ngồi” có $n = 3$ từ, với $d_{model} = 4$ (nhỏ để dễ hình dung) và $d_k = 2$:

$$\underbrace{X}_{3 \times 4} \times \underbrace{W^Q}_{4 \times 2} = \underbrace{Q}_{3 \times 2}$$

$$\begin{pmatrix} - & \text{Con} & - & - \\ - & \text{mèo} & - & - \\ - & \text{ngồi} & - & - \end{pmatrix}_{3 \times 4} \times \begin{pmatrix} \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \end{pmatrix}_{4 \times 2} = \begin{pmatrix} q_{1,1} & q_{1,2} \\ q_{2,1} & q_{2,2} \\ q_{3,1} & q_{3,2} \end{pmatrix}_{3 \times 2}$$

Ý nghĩa: Mỗi từ ban đầu là vector 4 chiều, sau phép chiếu W^Q , nó trở thành vector Query chỉ còn 2 chiều — nhưng 2 chiều này mã hoá thông tin “tôi đang tìm gì?” của từ đó.

Mẹo: Phép tương tự: Thư viện sách

Hãy tưởng tượng bạn vào thư viện tìm sách:

- **Query (Q):** Câu hỏi bạn đặt ra — “Tôi muốn tìm sách về mèo”.
- **Key (K):** Nhãn/thẻ trên mỗi cuốn sách — “Sách này về động vật”, “Sách này về nấu ăn”, ...
- **Value (V):** Nội dung thực sự bên trong cuốn sách.

Bạn so khớp Query với từng Key (tính QK^T) để xem sách nào liên quan nhất, rồi đọc nội dung (Value) của những sách đó, ưu tiên sách có Key khớp nhất.

d_k chính là “số tiêu chí so khớp” (ví dụ: chủ đề, thể loại, năm xuất bản, ...). Càng nhiều tiêu chí, so khớp càng chính xác nhưng cũng tốn tài nguyên hơn.

2.2.2 Công thức Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (13)$$

Ghi chú: Tại sao chia cho $\sqrt{d_k}$?

Khi d_k lớn, tích vô hướng $q_i^T k_j$ có xu hướng tăng theo d_k , khiến softmax bão hoà (các giá trị sau softmax gần 0 hoặc 1), làm gradient rất nhỏ. Chia cho $\sqrt{d_k}$ giúp ổn định phân phối và cải thiện quá trình huấn luyện. Cụ thể: nếu q, k có các thành phần i.i.d. với trung bình 0 và phương sai 1, thì $\text{Var}(q^T k) = d_k$. Chia cho $\sqrt{d_k}$ đưa phương sai về 1.

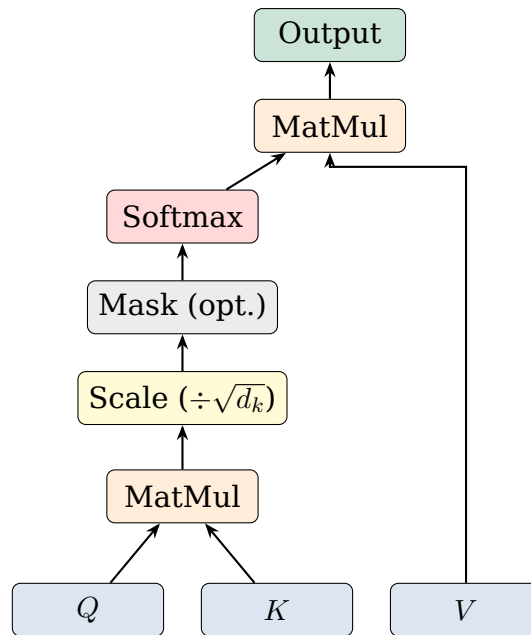


Figure 4: Sơ đồ Scaled Dot-Product Attention.

2.2.3 Ví dụ tính toán cụ thể

Ví dụ: Tính Attention bằng tay

Giả sử ta có 3 từ với $d_k = 2$:

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

Bước 1: Tính QK^T :

$$QK^T = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & 1 \end{pmatrix}$$

Bước 2: Chia cho $\sqrt{d_k} = \sqrt{2} \approx 1.414$:

$$\frac{QK^T}{\sqrt{2}} \approx \begin{pmatrix} 0.707 & 0 & 0.707 \\ 0.707 & 0.707 & 0 \\ 1.414 & 0.707 & 0.707 \end{pmatrix}$$

Bước 3: Áp dụng softmax theo từng hàng (ví dụ hàng 1):

$$\text{softmax}(0.707, 0, 0.707) = (0.387, 0.226, 0.387)$$

Bước 4: Nhân với V để được output. Mỗi hàng output là tổ hợp có trọng số của các hàng trong V .

3 Multi-Head Attention (Attention đa đầu)

3.1 Mục đích và kiến trúc

Thay vì chỉ tính attention một lần, ta thực hiện **song song** h lần với các phép chiếu khác nhau. Mỗi “đầu” (head) có thể học được các mối quan hệ khác nhau.

Định nghĩa: Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (14)$$

$$\text{trong đó } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (15)$$

với $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Thường: $d_k = d_v = d_{\text{model}}/h$. Ví dụ: $d_{\text{model}} = 512, h = 8 \Rightarrow d_k = d_v = 64$.

Ví dụ: Nhiều góc nhìn

Với câu: “Ngân hàng bên bờ sông rất đẹp”:

- **Head 1** có thể học: “ngân hàng” → “bờ sông” (quan hệ vị trí)
- **Head 2** có thể học: “đẹp” → “ngân hàng bên bờ sông” (quan hệ ngữ nghĩa)
- **Head 3** có thể học: “bên” → “bờ sông” (quan hệ cú pháp)

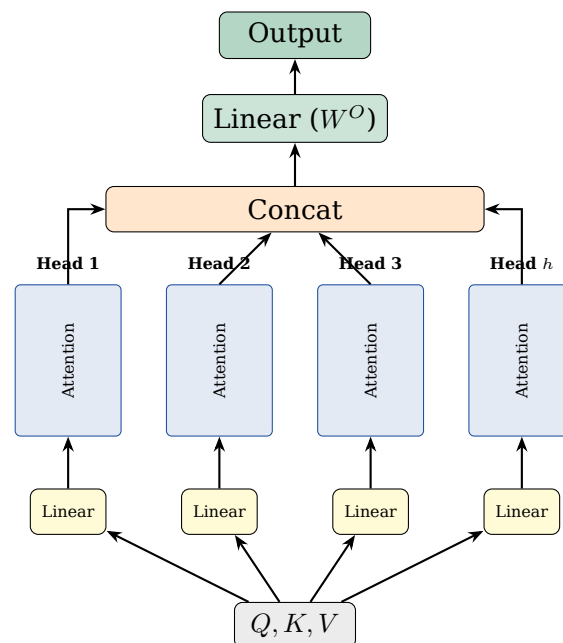


Figure 5: Kiến trúc Multi-Head Attention: h đầu attention hoạt động song song, sau đó nối (concat) và chiếu tuyến tính.

3.2 Chi tiết triển khai

Trong thực tế, thay vì tạo h ma trận riêng biệt, ta dùng **một phép chiếu lớn** rồi chia (reshape) thành h đầu:

```

1 # X: (batch, seq_len, d_model)
2 # Chieu thanh Q, K, V: (batch, seq_len, d_model)
3 Q = linear_q(X)
4 K = linear_k(X)
5 V = linear_v(X)
6
7 # Reshape: (batch, seq_len, h, d_k) -> (batch, h, seq_len, d_k)
8 Q = Q.view(batch, seq_len, h, d_k).transpose(1, 2)
9 K = K.view(batch, seq_len, h, d_k).transpose(1, 2)
10 V = V.view(batch, seq_len, h, d_v).transpose(1, 2)

```

Listing 1: Multi-Head Attention — Ý tưởng reshape

4 Kiến trúc Transformer

Transformer được giới thiệu trong bài báo nổi tiếng “Attention Is All You Need” (Vaswani et al., 2017). Đây là kiến trúc **hoàn toàn dựa trên attention**, loại bỏ RNN và CNN.

4.1 Cấu trúc Encoder-Decoder

Ghi chú: Thông số mô hình gốc

Trong bài báo gốc: $N = 6$ lớp, $d_{model} = 512$, $h = 8$ đầu attention, $d_{ff} = 2048$ (kích thước FFN ẩn), $d_k = d_v = 64$.

4.2 Positional Encoding (Mã hoá vị trí)

Transformer xử lý tất cả vị trí **đồng thời** (không tuần tự như RNN), nên cần thêm thông tin vị trí. Bài báo gốc dùng hàm sin/cos:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (16)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (17)$$

trong đó pos là vị trí trong câu và i là chỉ số chiều.

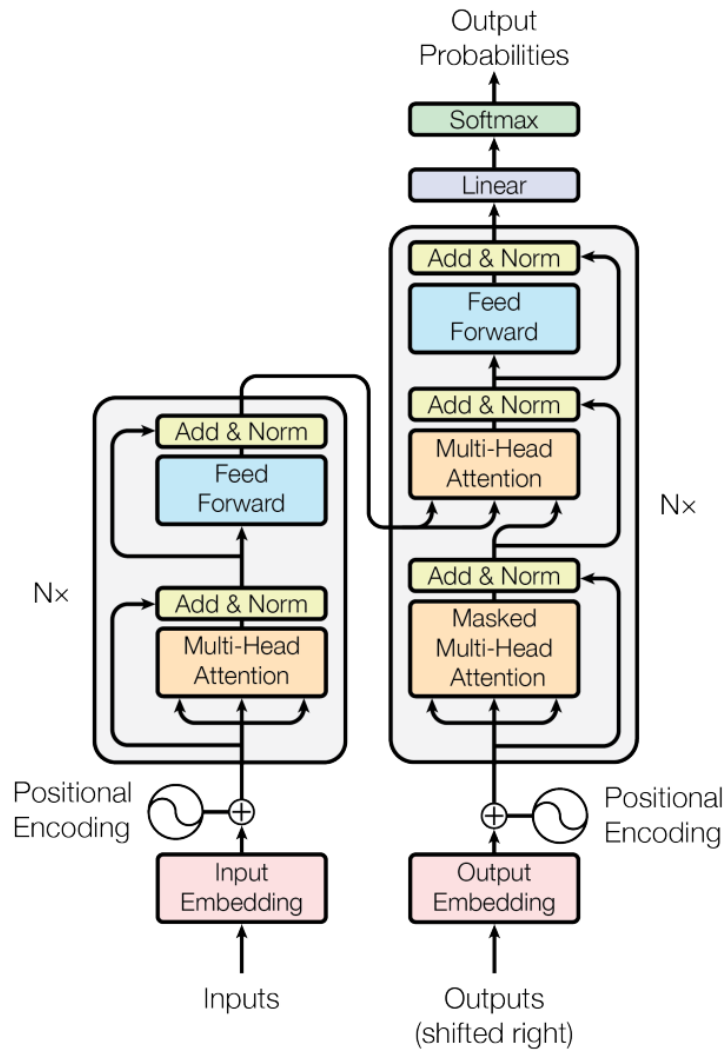


Figure 1: The Transformer - model architecture.

Figure 6: Kiến trúc tổng quan Transformer.

Ví dụ: Trực giác Positional Encoding

Hãy tưởng tượng mỗi vị trí được mã hoá như thời gian trên đồng hồ:

- Chiều thấp (i nhỏ): dao động nhanh, phân biệt các vị trí gần nhau (như kim giây)
- Chiều cao (i lớn): dao động chậm, mã hoá vị trí tổng thể (như kim giờ)

Ưu điểm: mô hình có thể suy luận về khoảng cách tương đối giữa hai vị trí, vì PE_{pos+k} có thể biểu diễn bằng phép biến đổi tuyến tính của PE_{pos} .

Ví dụ: Tính PE bằng tay — $d_{model} = 4$

Giả sử $d_{model} = 4$ (mỗi từ biểu diễn bằng vector 4 chiều). Ta có 4 chiều: chỉ số $i = 0, 1$ (vì $2i$ và $2i + 1$ cho ra chiều 0, 1, 2, 3).

Nhắc lại công thức:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (\text{chiều chẵn: } 0, 2, 4, \dots)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (\text{chiều lẻ: } 1, 3, 5, \dots)$$

Bước 1: Tính mẫu số cho từng cặp chiều.

Với $d_{model} = 4$:

$$i = 0 : \quad 10000^{2 \times 0 / 4} = 10000^0 = \mathbf{1} \quad (\text{chiều 0 và 1})$$

$$i = 1 : \quad 10000^{2 \times 1 / 4} = 10000^{0.5} = \sqrt{10000} = \mathbf{100} \quad (\text{chiều 2 và 3})$$

Bước 2: Tính PE cho từng vị trí.

Vị trí $pos = 0$ (từ đầu tiên):

$$PE_{(0,0)} = \sin(0/1) = \sin(0) = \mathbf{0}$$

$$PE_{(0,1)} = \cos(0/1) = \cos(0) = \mathbf{1}$$

$$PE_{(0,2)} = \sin(0/100) = \sin(0) = \mathbf{0}$$

$$PE_{(0,3)} = \cos(0/100) = \cos(0) = \mathbf{1}$$

$$\Rightarrow PE_0 = (0, 1, 0, 1)$$

Vị trí $pos = 1$ (từ thứ hai):

$$PE_{(1,0)} = \sin(1/1) = \sin(1) = \mathbf{0.841}$$

$$PE_{(1,1)} = \cos(1/1) = \cos(1) = \mathbf{0.540}$$

$$PE_{(1,2)} = \sin(1/100) = \sin(0.01) = \mathbf{0.010}$$

$$PE_{(1,3)} = \cos(1/100) = \cos(0.01) = \mathbf{1.000}$$

$$\Rightarrow PE_1 = (0.841, 0.540, 0.010, 1.000)$$

Vị trí $pos = 2$ (từ thứ ba):

$$PE_{(2,0)} = \sin(2/1) = \sin(2) = \mathbf{0.909}$$

$$PE_{(2,1)} = \cos(2/1) = \cos(2) = \mathbf{-0.416}$$

$$PE_{(2,2)} = \sin(2/100) = \sin(0.02) = \mathbf{0.020}$$

$$PE_{(2,3)} = \cos(2/100) = \cos(0.02) = \mathbf{1.000}$$

$$\Rightarrow PE_2 = (0.909, -0.416, 0.020, 1.000)$$

Vị trí $pos = 3$ (từ thứ tư):

$$PE_{(3,0)} = \sin(3) = \mathbf{0.141}$$

$$PE_{(3,1)} = \cos(3) = \mathbf{-0.990}$$

$$PE_{(3,2)} = \sin(0.03) = \mathbf{0.030}$$

$$PE_{(3,3)} = \cos(0.03) = \mathbf{1.000}$$

$$\Rightarrow PE_3 = (0.141, -0.990, 0.030, 1.000)_{16}$$

Ví dụ: Tổng hợp — Bảng Positional Encoding

Ma trận PE hoàn chỉnh cho 4 vị trí, $d_{model} = 4$:

	Chiều 0 $\sin(pos/1)$	Chiều 1 $\cos(pos/1)$	Chiều 2 $\sin(pos/100)$	Chiều 3 $\cos(pos/100)$	
$pos = 0$	0.000	1.000	0.000	1.000	“Con”
$pos = 1$	0.841	0.540	0.010	1.000	“mèo”
$pos = 2$	0.909	-0.416	0.020	1.000	“ngồi”
$pos = 3$	0.141	-0.990	0.030	1.000	“trên”

Quan sát quan trọng:

- **Chiều 0-1** (mẫu số = 1): dao động **rất nhanh** — thay đổi mạnh giữa $pos = 0, 1, 2, 3$. Giống kim giây trên đồng hồ.
- **Chiều 2-3** (mẫu số = 100): dao động **rất chậm** — gần như không đổi ($0.000 \rightarrow 0.010 \rightarrow 0.020 \rightarrow 0.030$). Giống kim giờ trên đồng hồ.
- Mỗi vị trí có một “mã vạch” (barcode) **duy nhất** — không có hai vị trí nào có vector PE giống nhau.

Ví dụ: Bước cuối — Cộng PE vào Embedding

Giả sử sau bước embedding, câu “Con mèo ngồi trên” được biểu diễn thành ma trận X (mỗi hàng là vector embedding 4 chiều của một từ):

$$X = \begin{pmatrix} 0.2 & 0.5 & -0.1 & 0.3 \\ 0.8 & -0.3 & 0.6 & 0.1 \\ 0.1 & 0.7 & -0.4 & 0.9 \\ -0.2 & 0.4 & 0.5 & -0.1 \end{pmatrix} \quad (\text{Embedding})$$

Cộng thêm PE:

$$\begin{aligned} X + PE &= \begin{pmatrix} 0.2 + 0.000 & 0.5 + 1.000 & -0.1 + 0.000 & 0.3 + 1.000 \\ 0.8 + 0.841 & -0.3 + 0.540 & 0.6 + 0.010 & 0.1 + 1.000 \\ 0.1 + 0.909 & 0.7 - 0.416 & -0.4 + 0.020 & 0.9 + 1.000 \\ -0.2 + 0.141 & 0.4 - 0.990 & 0.5 + 0.030 & -0.1 + 1.000 \end{pmatrix} \\ &= \begin{pmatrix} 0.200 & 1.500 & -0.100 & 1.300 \\ 1.641 & 0.240 & 0.610 & 1.100 \\ 1.009 & 0.284 & -0.380 & 1.900 \\ -0.059 & -0.590 & 0.530 & 0.900 \end{pmatrix} \quad (\text{Embedding} + \text{vị trí}) \end{aligned}$$

Bây giờ mỗi vector từ đã chứa **cả nghĩa của từ** (embedding) **lẫn vị trí** của nó trong câu. Từ “mèo” ở vị trí 1 sẽ có vector khác với từ “mèo” ở vị trí 5, nhờ PE khác nhau.

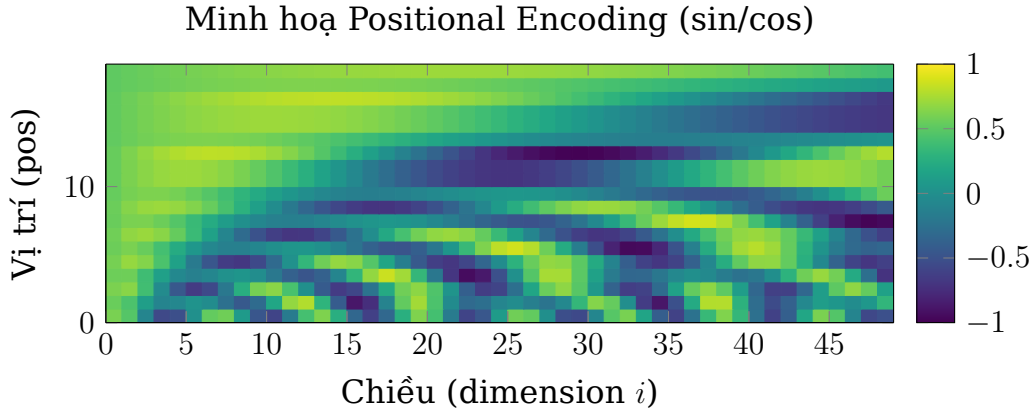


Figure 7: Heatmap minh hoạ giá trị positional encoding: trục hoành là chiều, trục tung là vị trí.

4.3 Feed-Forward Networks (FFN)

Mỗi lớp Transformer chứa một mạng FFN áp dụng **theo từng vị trí** (position-wise):

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (18)$$

trong đó $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

Ghi chú: Vai trò FFN

FFN đóng vai trò “bộ nhớ” lưu trữ tri thức thế giới. Attention tìm mối quan hệ giữa các token; FFN biến đổi biểu diễn bằng kiến thức đã học. Kích thước $d_{\text{ff}} = 4 \times d_{\text{model}}$ là quy ước phổ biến.

4.4 Layer Normalization và Residual Connections

4.4.1 Residual Connection (Kết nối tắt)

$$\text{output} = x + \text{Sublayer}(x) \quad (19)$$

Giúp gradient lan truyền dễ dàng qua nhiều lớp, tránh vanishing gradient.

4.4.2 Layer Normalization

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (20)$$

trong đó μ, σ^2 là trung bình và phương sai tính trên chiều feature của x , còn γ, β là tham số học được.

Kết hợp lại, mỗi sub-layer trong Transformer thực hiện:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (21)$$

4.5 Ví dụ tổng hợp: Transformer dịch một câu từ đầu đến cuối

Phần này sẽ đi qua **từng bước** cách Transformer dịch câu:

“Tôi yêu Việt Nam” $\rightarrow$ *“I love Vietnam”*

Ta dùng mô hình nhỏ để dễ tính tay: $d_{model} = 4$, $h = 2$ head, $d_k = d_v = 2$, $d_{ff} = 8$, $N = 1$ lớp encoder/decoder. Từ vựng nguồn (tiếng Việt) có 5000 từ, từ vựng đích (tiếng Anh) có 4000 từ.

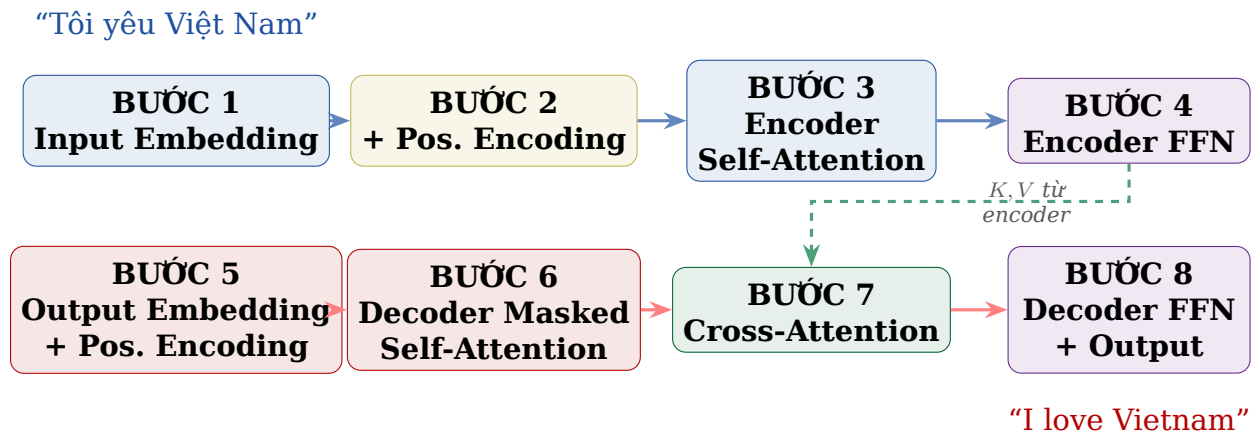


Figure 8: Tổng quan 8 bước Transformer xử lý câu dịch. Hàng trên: Encoder. Hàng dưới: Decoder.

4.5.1 Bước 1: Input Embedding — Chuyển từ thành vector

Mỗi từ trong câu nguồn được tra bảng embedding (lookup table) để lấy vector $d_{model} = 4$ chiều. Bảng embedding có kích thước 5000×4 (5000 từ vựng, mỗi từ 4 chiều), được **học** trong quá trình huấn luyện.

Ví dụ: Embedding câu nguồn

Giả sử các từ có chỉ số trong từ điển: “Tôi” = 12, “yêu” = 345, “Việt” = 1002, “Nam” = 503.

Tra bảng embedding (giá trị giả để minh hoạ):

$$X_{emb} = \begin{pmatrix} 0.21 & -0.45 & 0.78 & 0.12 \\ 0.65 & 0.33 & -0.22 & 0.89 \\ -0.11 & 0.72 & 0.55 & -0.38 \\ 0.44 & 0.18 & 0.61 & -0.05 \end{pmatrix} \begin{matrix} \leftarrow \text{“Tôi”} \\ \leftarrow \text{“yêu”} \\ \leftarrow \text{“Việt”} \\ \leftarrow \text{“Nam”} \end{matrix}$$

Kích thước: 4×4 (4 từ, mỗi từ 4 chiều).

Nhân với $\sqrt{d_{model}}$: Theo bài báo gốc, embedding được nhân với $\sqrt{4} = 2$ để cân bằng scale với positional encoding:

$$X_{scaled} = 2 \times X_{emb} = \begin{pmatrix} 0.42 & -0.90 & 1.56 & 0.24 \\ 1.30 & 0.66 & -0.44 & 1.78 \\ -0.22 & 1.44 & 1.10 & -0.76 \\ 0.88 & 0.36 & 1.22 & -0.10 \end{pmatrix}$$

4.5.2 Bước 2: Cộng Positional Encoding

Vì Transformer xử lý tất cả từ cùng lúc, ta cần thêm thông tin vị trí (đã học tính ở phần 4).

Ví dụ: Cộng PE vào embedding

Dùng lại PE đã tính (với $d_{model} = 4$):

$$PE = \begin{pmatrix} 0.000 & 1.000 & 0.000 & 1.000 \\ 0.841 & 0.540 & 0.010 & 1.000 \\ 0.909 & -0.416 & 0.020 & 1.000 \\ 0.141 & -0.990 & 0.030 & 1.000 \end{pmatrix}$$

Đầu vào encoder:

$$X_{enc} = X_{scaled} + PE = \begin{pmatrix} 0.420 & 0.100 & 1.560 & 1.240 \\ 2.141 & 1.200 & -0.430 & 2.780 \\ 0.689 & 1.024 & 1.120 & 0.240 \\ 1.021 & -0.630 & 1.250 & 0.900 \end{pmatrix}$$

Mỗi hàng bây giờ mã hoá **cả nghĩa lẫn vị trí** của từ. Đây chính là đầu vào cho Encoder.

4.5.3 Bước 3: Encoder Self-Attention — “Mỗi từ nhìn tất cả từ khác”

Đây là bước quan trọng nhất của Encoder. Với $h = 2$ head và $d_k = 2$:

Ví dụ: Encoder Self-Attention chi tiết**3a. Chiếu X_{enc} thành Q, K, V cho Head 1:**

Ma trận trọng số W_1^Q, W_1^K, W_1^V (mỗi cái 4×2) được học. Giả sử sau phép nhân:

$$Q_1 = X_{enc}W_1^Q = \begin{pmatrix} 0.5 & 1.2 \\ 1.8 & 0.3 \\ 0.9 & 0.7 \\ 0.4 & 1.5 \end{pmatrix}, K_1 = \begin{pmatrix} 0.3 & 0.8 \\ 1.5 & 0.2 \\ 0.6 & 1.1 \\ 0.7 & 0.9 \end{pmatrix}, V_1 = \begin{pmatrix} 0.2 & 0.6 \\ 1.1 & 0.4 \\ 0.5 & 0.9 \\ 0.8 & 0.3 \end{pmatrix}$$

3b. Tính điểm attention $Q_1K_1^T/\sqrt{d_k}$:

$$Q_1K_1^T = \begin{pmatrix} 0.5 \times 0.3 + 1.2 \times 0.8 & 0.5 \times 1.5 + 1.2 \times 0.2 & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

Giả sử kết quả sau chia $\sqrt{2} \approx 1.414$:

$$\frac{Q_1K_1^T}{\sqrt{2}} \approx \begin{pmatrix} 0.78 & 0.70 & 1.15 & 1.01 \\ 0.55 & \mathbf{1.95} & 1.01 & 1.10 \\ 0.66 & 1.12 & 0.93 & 0.88 \\ 0.89 & 0.72 & 1.40 & \mathbf{1.25} \end{pmatrix} \begin{matrix} \leftarrow \text{Tôi hỏi: liên quan gì?} \\ \leftarrow \text{yêu hỏi: liên quan gì?} \\ \leftarrow \text{Việt hỏi: liên quan gì?} \\ \leftarrow \text{Nam hỏi: liên quan gì?} \end{matrix}$$

Đọc hàng 2 (từ “yêu”): điểm cao nhất là cột 2 (1.95) — “yêu” chú ý mạnh nhất vào chính nó (hợp lý, vì “yêu” là động từ chính).

Đọc hàng 4 (từ “Nam”): điểm cao nhất là cột 3 (1.40) — “Nam” chú ý mạnh vào “Việt” (hợp lý, vì “Việt Nam” là một cụm từ).

3c. Softmax theo hàng $\rightarrow$ trọng số attention:

$$\alpha_1 = \text{softmax}\left(\frac{Q_1K_1^T}{\sqrt{2}}\right) \approx \begin{pmatrix} 0.21 & 0.19 & \mathbf{0.31} & 0.27 \\ 0.13 & \mathbf{0.53} & 0.21 & 0.23 \\ 0.19 & 0.30 & 0.25 & 0.24 \\ 0.21 & 0.18 & \mathbf{0.35} & 0.25 \end{pmatrix}$$

3d. Nhân với V_1 để lấy output (tổ hợp có trọng số của Value):

$$\text{head}_1 = \alpha_1 \times V_1 = \begin{pmatrix} 0.65 & 0.57 \\ 0.80 & 0.48 \\ 0.69 & 0.54 \\ 0.58 & 0.62 \end{pmatrix} \quad (\text{kích thước: } 4 \times 2)$$

3e. Head 2 làm tương tự với W_2^Q, W_2^K, W_2^V khác $\rightarrow \text{head}_2$ (kích thước 4×2).

Head 2 có thể học được quan hệ khác, ví dụ: “Việt” chú ý mạnh vào “Nam” (quan hệ cú pháp).

3f. Nối 2 head và chiếu:

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \text{head}_2) \times W^O = \underbrace{[\text{head}_1 \mid \text{head}_2]}_{4 \times 4} \times \underbrace{W^O}_{4 \times 4}$$

Kết quả: ma trận 4×4 — mỗi từ vẫn là vector 4 chiều (d_{model}), nhưng giờ đã **chứa thông tin từ tất cả các từ khác** trong câu.

4.5.4 Bước 3.5: Add & Norm — Residual + Layer Normalization

Ví dụ: Add & Norm sau Self-Attention

Gọi output của Multi-Head Attention là Z :

$$X_{after_attn} = \text{LayerNorm}\left(\underbrace{X_{enc}}_{\text{đầu vào gốc}} + \underbrace{Z}_{\text{output attention}}\right)$$

Tại sao cộng X_{enc} gốc? Đây là residual connection — giúp gradient chảy trực tiếp qua, tránh vanishing gradient. Hiểu đơn giản: “giữ lại thông tin gốc, cộng thêm thông tin mới từ attention”.

LayerNorm chuẩn hoá mỗi vector (mỗi hàng) để có trung bình ≈ 0 và phương sai ≈ 1 , giúp huấn luyện ổn định.

4.5.5 Bước 4: Encoder FFN — Biến đổi phi tuyến

Ví dụ: Feed-Forward Network

FFN áp dụng **cho từng từ riêng biệt** (position-wise), không trộn thông tin giữa các từ:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Với $d_{model} = 4$ và $d_{ff} = 8$:

- W_1 : 4×8 — “mở rộng” mỗi vector từ 4 chiều lên 8 chiều.
- ReLU: bỏ các giá trị âm (đặt thành 0).
- W_2 : 8×4 — “nén” lại về 4 chiều.

Ví dụ cho vector của từ “yêu” ($x = [0.80, 0.48, \dots]$, 4 chiều):

$$\underbrace{x}_{1 \times 4} \xrightarrow{W_1} \underbrace{[2.1, -0.3, 1.5, 0.0, 0.8, -1.2, 0.4, 1.9]}_{1 \times 8} \xrightarrow{\text{ReLU}} \underbrace{[2.1, 0, 1.5, 0, 0.8, 0, 0.4, 1.9]}_{1 \times 8} \xrightarrow{W_2} \underbrace{[0.92, 0.61, -0.15]}_{1 \times 4}$$

Sau Add & Norm lần 2:

$$X_{enc_out} = \text{LayerNorm}(X_{after_attn} + \text{FFN}(X_{after_attn}))$$

Đây là output cuối cùng của Encoder! Ma trận 4×4 — mỗi từ tiếng Việt giờ được biểu diễn bằng vector 4 chiều chứa đầy đủ ngữ cảnh từ toàn bộ câu. X_{enc_out} sẽ được gửi sang Decoder.

PHẦN DECODER — Sinh câu tiếng Anh từng từ một

Decoder hoạt động **tuần tự** (autoregressive): sinh từng từ một, từ trái sang phải. Mỗi bước dùng *các từ đã sinh trước đó + thông tin từ encoder* để dự đoán từ tiếp theo.

4.5.6 Bước 5: Output Embedding + Positional Encoding (Decoder Input)

Ví dụ: Decoder Input — Bước dịch $t = 1$ (sinh từ đầu tiên)

Khi bắt đầu, decoder nhận token đặc biệt <SOS> (Start of Sentence):

$$Y_0 = \text{Embedding}(\text{<SOS>}) \times \sqrt{d_{\text{model}}} + PE_0$$

Giả sử:

$$Y_0 = \underbrace{(0.30, -0.15, 0.62, 0.45)}_{\text{embedding <SOS>} \times 2} + \underbrace{(0, 1, 0, 1)}_{PE_0} = (0.30, 0.85, 0.62, 1.45)$$

Đây là ma trận 1×4 (chỉ 1 token).

Ví dụ: Decoder Input — Bước dịch $t = 3$ (đang sinh từ thứ 3)

Nếu decoder đã sinh “I” và “love”, input cho bước tiếp theo là:

$$Y_{in} = \begin{pmatrix} \text{Emb}(\text{<SOS>}) + PE_0 \\ \text{Emb}(\text{I}) + PE_1 \\ \text{Emb}(\text{love}) + PE_2 \end{pmatrix} = \begin{pmatrix} 0.30 & 0.85 & 0.62 & 1.45 \\ 1.52 & 0.91 & 0.33 & 1.72 \\ 0.75 & -0.12 & 1.48 & 0.63 \end{pmatrix} \quad (3 \times 4)$$

4.5.7 Bước 6: Decoder Masked Self-Attention — “Không được nhìn tương lai”

Ví dụ: Masked Self-Attention ở bước $t = 3$

Decoder tính self-attention giữa 3 token (<SOS>, “I”, “love”) giống hệt encoder, **nhưng** áp dụng causal mask:

6a. Tính $Q_d K_d^T / \sqrt{d_k}$ (giả sử):

$$\text{Scores} = \begin{pmatrix} 1.2 & 0.8 & 0.5 \\ 0.6 & 1.5 & 0.9 \\ 0.3 & 1.1 & 1.8 \end{pmatrix} \begin{array}{l} \leftarrow \text{<SOS>} \\ \leftarrow \text{“I”} \\ \leftarrow \text{“love”} \end{array}$$

6b. Áp dụng causal mask — thay vị trí “tương lai” bằng $-\infty$:

$$\text{Masked} = \begin{pmatrix} 1.2 & -\infty & -\infty \\ 0.6 & 1.5 & -\infty \\ 0.3 & 1.1 & 1.8 \end{pmatrix}$$

6c. Softmax theo hàng:

Hàng 1: $\text{softmax}(1.2, -\infty, -\infty) = (\mathbf{1.00}, 0.00, 0.00)$

→ <SOS> chỉ nhìn thấy chính nó

Hàng 2: $\text{softmax}(0.6, 1.5, -\infty) = (0.29, \mathbf{0.71}, 0.00)$

→ “I” nhìn <SOS> và chính nó, không nhìn “love”

Hàng 3: $\text{softmax}(0.3, 1.1, 1.8) = (0.12, 0.26, \mathbf{0.52})$

→ “love” nhìn tất cả 3 từ trước nó (kể cả chính nó)

6d. Nhân với $V_d \rightarrow \text{Add \& Norm} \rightarrow Y_{\text{masked}}$ (kích thước 3×4).

4.5.8 Bước 7: Cross-Attention — “Decoder hỏi Encoder”

Đây là bước **kết nối** giữa encoder và decoder — nơi decoder “tham khảo” câu nguồn tiếng Việt.

Định nghĩa: Cross-Attention: Q từ Decoder, K và V từ Encoder

- **Query (Q)**: đến từ **decoder** — “Tôi (decoder) đang cần thông tin gì?”
- **Key (K)**: đến từ **encoder** — “Mỗi từ tiếng Việt chứa thông tin gì?”
- **Value (V)**: đến từ **encoder** — “Đây là nội dung thực sự của từng từ tiếng Việt.”

Ví dụ: Cross-Attention chi tiết ở bước $t = 3$ **7a.** Chiếu:

$$Q_{cross} = Y_{masked} \times W_{cross}^Q \quad (\text{từ decoder: } 3 \times 2)$$

$$K_{cross} = X_{enc_out} \times W_{cross}^K \quad (\text{từ encoder: } 4 \times 2)$$

$$V_{cross} = X_{enc_out} \times W_{cross}^V \quad (\text{từ encoder: } 4 \times 2)$$

Chú ý: Q có 3 hàng (3 token decoder: <SOS>, “I”, “love”), còn K và V có 4 hàng (4 từ encoder: “Tôi”, “yêu”, “Việt”, “Nam”).

7b. Tính ma trận attention $Q_{cross} K_{cross}^T / \sqrt{d_k}$:

$$\text{Scores}_{cross} = \begin{pmatrix} 0.3 & 0.5 & 0.1 & \downarrow 0.2 \\ 0.2 & \mathbf{0.4} & 0.1 & 0.1 \\ 0.1 & \mathbf{0.9} & 0.3 & 0.2 \end{pmatrix} \begin{matrix} \text{encoder} \\ \text{Tôi} \\ \text{yêu} \\ \text{Việt} \\ \text{Nam} \end{matrix}$$

Không cần causal mask ở cross-attention! Decoder được phép nhìn toàn bộ câu nguồn.

7c. Softmax theo hàng:

$$\alpha_{cross} \approx \begin{pmatrix} 0.22 & \mathbf{0.30} & 0.21 & 0.27 \\ 0.24 & \mathbf{0.29} & 0.23 & 0.24 \\ 0.15 & \mathbf{0.46} & 0.22 & 0.17 \end{pmatrix}$$

Đọc hàng 3 (vị trí “love” trong decoder): trọng số cao nhất rơi vào cột 2 ($\alpha = 0.46$) — ứng với từ “yêu” trong encoder. **Hoàn toàn hợp lý!** Khi decoder đang xử lý “love”, nó cần chú ý nhiều nhất vào “yêu” (nghĩa tương đương).

7d. Nhân với V_{cross} (từ encoder):

$$\text{CrossAttn_out} = \alpha_{cross} \times V_{cross} \quad (3 \times 2 \text{ cho mỗi head})$$

Concat 2 head + $W^O \rightarrow 3 \times 4 \rightarrow \text{Add \& Norm.}$

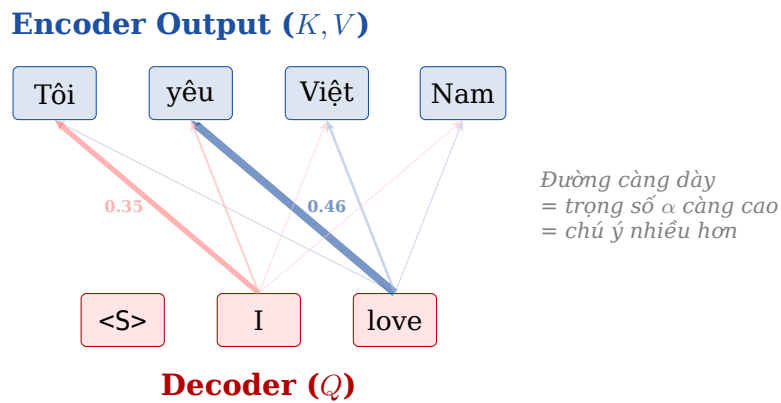


Figure 9: Cross-Attention: từ “love” (decoder) chú ý mạnh nhất vào “yêu” (encoder), từ “I” chú ý mạnh nhất vào “Tôi”.

4.5.9 Bước 8: Decoder FFN + Linear + Softmax — Sinh từ tiếp theo

Ví dụ: Sinh từ “Vietnam” ở bước $t = 3$

8a. FFN: Giống encoder FFN — mở rộng lên $d_{ff} = 8$ rồi nén về $d_{model} = 4$. Add & Norm.

Gọi output là D_{out} (kích thước 3×4). Ta chỉ quan tâm **hàng cuối cùng** (vị trí cuối = vị trí đang sinh từ mới):

$$d_3 = D_{out}[3, :] = (0.85, -0.23, 1.41, 0.67) \quad (\text{vector 4 chiều})$$

8b. Linear layer: Chiếu vector 4 chiều lên toàn bộ từ vựng tiếng Anh (4000 từ):

$$\text{logits} = d_3 \times W_{vocab} + b_{vocab} \quad (1 \times 4) \times (4 \times 4000) = (1 \times 4000)$$

8c. Softmax: Chuyển 4000 logits thành xác suất:

$$P(y_3 \mid y_{<3}, X) = \text{softmax}(\text{logits}) \in \mathbb{R}^{4000}$$

Giả sử kết quả:

Từ	Xác suất
Vietnam	0.82
country	0.05
Hanoi	0.03
Asia	0.02
...(3996 từ khác)	...

Từ có xác suất cao nhất: **“Vietnam”** ($P = 0.82$) $\Rightarrow$ decoder chọn “Vietnam” làm từ tiếp theo.

4.5.10 Tổng kết: Toàn bộ quá trình dịch

Ví dụ: Timeline dịch hoàn chỉnh

Bước t	Input decoder	Cross-Attn tập trung vào	Output
$t = 1$	<SOS>	"Tôi" ($\alpha = 0.45$)	"I"
$t = 2$	<SOS>, I	"yêu" ($\alpha = 0.52$)	"love"
$t = 3$	<SOS>, I, love	"Việt"+"Nam" ($\alpha = 0.35, 0.30$)	"Vietnam"
$t = 4$	<SOS>, I, love, Vietnam	(kết thúc câu)	<EOS>

Khi decoder sinh token <EOS> (End of Sentence), quá trình dịch kết thúc.

Kết quả: **"I love Vietnam"** ✓

Mẹo: Tóm tắt luồng dữ liệu

1. **Encoder** xử lý *một lần* toàn bộ câu nguồn $\rightarrow X_{enc_out}$ (chứa ngữ cảnh đầy đủ).
2. **Decoder** chạy *nhiều bước* ($t = 1, 2, 3, \dots$), mỗi bước:
 - a. Nhận các từ đã sinh + PE $\rightarrow$ Masked Self-Attention (không nhìn tương lai).
 - b. Cross-Attention: dùng Query từ decoder, Key/Value từ encoder $\rightarrow$ "hỏi" câu nguồn.
 - c. FFN $\rightarrow$ Linear $\rightarrow$ Softmax $\rightarrow$ chọn từ có xác suất cao nhất.
3. Lặp lại cho đến khi sinh <EOS>.

5 Xây dựng Transformer Block

5.1 Encoder Block

Mỗi encoder block gồm hai sub-layer:

1. **Multi-Head Self-Attention**: mỗi token chú ý đến tất cả token trong chuỗi.
2. **Feed-Forward Network**: biến đổi phi tuyến theo từng vị trí.

Mỗi sub-layer có residual connection + layer normalization.

```

1 import torch
2 import torch.nn as nn
3
4 class EncoderBlock(nn.Module):
5     def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
6         super().__init__()
7         self.self_attn = nn.MultiheadAttention(
8             d_model, n_heads, dropout=dropout, batch_first=True
9         )

```

```

10     self.ffn = nn.Sequential(
11         nn.Linear(d_model, d_ff),
12         nn.ReLU(),
13         nn.Linear(d_ff, d_model),
14     )
15     self.norm1 = nn.LayerNorm(d_model)
16     self.norm2 = nn.LayerNorm(d_model)
17     self.dropout = nn.Dropout(dropout)
18
19     def forward(self, x, src_mask=None):
20         # Self-Attention + Add & Norm
21         attn_out, _ = self.self_attn(x, x, x, attn_mask=src_mask)
22         x = self.norm1(x + self.dropout(attn_out))
23         # FFN + Add & Norm
24         ffn_out = self.ffn(x)
25         x = self.norm2(x + self.dropout(ffn_out))
26         return x

```

Listing 2: Encoder Block bằng PyTorch

5.2 Decoder Block

Decoder block có **ba** sub-layer:

1. **Masked Multi-Head Self-Attention**: giống encoder, nhưng dùng causal mask để token hiện tại không “nhìn” được các token tương lai (autoregressive).
2. **Cross-Attention**: Query từ decoder, Key và Value từ encoder output — cho phép decoder tham khảo thông tin từ câu nguồn.
3. **Feed-Forward Network**: giống encoder.

Định nghĩa: Causal Mask

Ma trận mask tam giác trên (upper triangular) với giá trị $-\infty$:

$$\text{Mask} = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Sau khi cộng vào $QK^T/\sqrt{d_k}$ rồi qua softmax, các vị trí $-\infty$ trở thành 0
 $\Rightarrow$ token i chỉ attend đến token $1, 2, \dots, i$.

```

1 class DecoderBlock(nn.Module):
2     def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
3         super().__init__()
4         self.masked_self_attn = nn.MultiheadAttention(
5             d_model, n_heads, dropout=dropout, batch_first=True
6         )
7         self.cross_attn = nn.MultiheadAttention(
8             d_model, n_heads, dropout=dropout, batch_first=True

```

```

9         )
10        self.ffn = nn.Sequential(
11            nn.Linear(d_model, d_ff),
12            nn.ReLU(),
13            nn.Linear(d_ff, d_model),
14        )
15        self.norm1 = nn.LayerNorm(d_model)
16        self.norm2 = nn.LayerNorm(d_model)
17        self.norm3 = nn.LayerNorm(d_model)
18        self.dropout = nn.Dropout(dropout)
19
20    def forward(self, x, enc_output, tgt_mask=None, memory_mask=
21    None):
22        # Masked Self-Attention
23        attn1, _ = self.masked_self_attn(
24            x, x, x, attn_mask=tgt_mask
25        )
26        x = self.norm1(x + self.dropout(attn1))
27        # Cross-Attention
28        attn2, _ = self.cross_attn(
29            x, enc_output, enc_output, attn_mask=memory_mask
30        )
31        x = self.norm2(x + self.dropout(attn2))
32        # FFN
33        ffn_out = self.ffn(x)
34        x = self.norm3(x + self.dropout(ffn_out))
35        return x

```

Listing 3: Decoder Block bằng PyTorch

6 Ứng dụng của Transformer

6.1 Xử lý ngôn ngữ tự nhiên (NLP)

Transformer đã cách mạng hoá toàn bộ lĩnh vực NLP:

Table 2: Các ứng dụng NLP chính của Transformer

Bài toán	Mô hình tiêu biểu	Mô tả
Dịch máy	Transformer gốc	Encoder-Decoder
Phân loại văn bản	BERT	Encoder-only
Sinh văn bản	GPT	Decoder-only
Tóm tắt văn bản	T5, BART	Encoder-Decoder
Hỏi đáp	BERT, RoBERTa	Encoder + fine-tuning
Nhận dạng thực thể	BERT-NER	Token classification

6.2 Vision Transformers (ViT)

Định nghĩa: Vision Transformer

ViT (Dosovitskiy et al., 2020) áp dụng Transformer cho bài toán phân loại ảnh bằng cách chia ảnh thành các “patch” (mảnh) và xem mỗi patch như một “token”.

Quy trình ViT:

1. Chia ảnh $H \times W$ thành các patch $P \times P$. Số patch: $N = HW/P^2$.
2. Mỗi patch được dàn phẳng (flatten) và chiếu tuyến tính thành vector d -chiều.
3. Thêm [CLS] token ở đầu, cộng positional embedding.
4. Đưa qua L lớp Transformer Encoder.
5. Dùng output tại vị trí [CLS] cho phân loại.

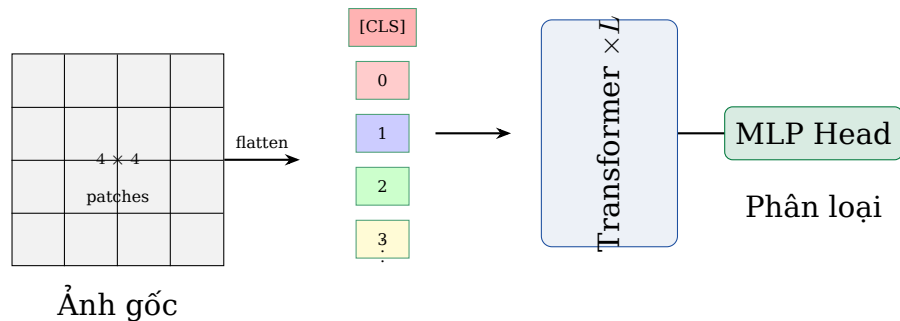


Figure 10: Kiến trúc Vision Transformer (ViT): ảnh được chia thành patch, thêm [CLS] token, đưa qua Transformer Encoder.

7 Triển khai Transformer đơn giản với PyTorch

Dưới đây là mã nguồn hoàn chỉnh xây dựng một Transformer từ đầu.

7.1 Positional Encoding

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class PositionalEncoding(nn.Module):
6     def __init__(self, d_model, max_len=5000, dropout=0.1):
7         super().__init__()
8         self.dropout = nn.Dropout(dropout)
9         pe = torch.zeros(max_len, d_model)
10        position = torch.arange(0, max_len).unsqueeze(1).float()
11        div_term = torch.exp(
12            torch.arange(0, d_model, 2).float()
13            * (-math.log(10000.0) / d_model)
14        )

```

```

15     pe[:, 0::2] = torch.sin(position * div_term)
16     pe[:, 1::2] = torch.cos(position * div_term)
17     pe = pe.unsqueeze(0) # (1, max_len, d_model)
18     self.register_buffer('pe', pe)
19
20     def forward(self, x):
21         # x: (batch, seq_len, d_model)
22         x = x + self.pe[:, :x.size(1), :]
23         return self.dropout(x)

```

Listing 4: Positional Encoding

7.2 Scaled Dot-Product Attention (Tự triển khai)

```

1 def scaled_dot_product_attention(Q, K, V, mask=None):
2     """
3     Q, K, V: (batch, heads, seq_len, d_k)
4     mask: (batch, 1, 1, seq_len) hoặc (batch, 1, seq_len, seq_len)
5     """
6     d_k = Q.size(-1)
7     scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
8     if mask is not None:
9         scores = scores.masked_fill(mask == 0, float('-inf'))
10    attn_weights = torch.softmax(scores, dim=-1)
11    output = torch.matmul(attn_weights, V)
12    return output, attn_weights

```

Listing 5: Scaled Dot-Product Attention từ đầu

7.3 Multi-Head Attention (Tự triển khai)

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         assert d_model % n_heads == 0
5         self.d_k = d_model // n_heads
6         self.n_heads = n_heads
7         self.W_q = nn.Linear(d_model, d_model)
8         self.W_k = nn.Linear(d_model, d_model)
9         self.W_v = nn.Linear(d_model, d_model)
10        self.W_o = nn.Linear(d_model, d_model)
11
12    def forward(self, Q, K, V, mask=None):
13        batch = Q.size(0)
14        # Chiều tuyến tính và reshape
15        Q = self.W_q(Q).view(batch, -1, self.n_heads, self.d_k).
16            transpose(1,2)
17        K = self.W_k(K).view(batch, -1, self.n_heads, self.d_k).
18            transpose(1,2)

```

```

17     V = self.W_v(V).view(batch, -1, self.n_heads, self.d_k).
        transpose(1,2)
18     # Tính attention
19     out, attn = scaled_dot_product_attention(Q, K, V, mask)
20     # Noi cac head va chieu output
21     out = out.transpose(1,2).contiguous().view(batch, -1, self
        .n_heads * self.d_k)
22     return self.W_o(out)

```

Listing 6: Multi-Head Attention từ đầu

7.4 Transformer Model hoàn chỉnh

```

1 class TransformerModel(nn.Module):
2     def __init__(self, src_vocab, tgt_vocab, d_model=512,
3                 n_heads=8, n_layers=6, d_ff=2048, dropout=0.1):
4         super().__init__()
5         self.d_model = d_model
6         # Embeddings
7         self.src_emb = nn.Embedding(src_vocab, d_model)
8         self.tgt_emb = nn.Embedding(tgt_vocab, d_model)
9         self.pos_enc = PositionalEncoding(d_model, dropout=dropout
10        )
11        # Encoder & Decoder
12        enc_layer = nn.TransformerEncoderLayer(
13            d_model, n_heads, d_ff, dropout, batch_first=True
14        )
15        self.encoder = nn.TransformerEncoder(enc_layer, n_layers)
16        dec_layer = nn.TransformerDecoderLayer(
17            d_model, n_heads, d_ff, dropout, batch_first=True
18        )
19        self.decoder = nn.TransformerDecoder(dec_layer, n_layers)
20        # Output
21        self.fc_out = nn.Linear(d_model, tgt_vocab)
22
23        def forward(self, src, tgt, src_mask=None, tgt_mask=None,
24                    memory_mask=None):
25            # Encoder
26            src = self.pos_enc(self.src_emb(src) * math.sqrt(self.
27                d_model))
28            memory = self.encoder(src, mask=src_mask)
29            # Decoder
30            tgt = self.pos_enc(self.tgt_emb(tgt) * math.sqrt(self.
31                d_model))
32            output = self.decoder(tgt, memory, tgt_mask=tgt_mask,
33                memory_mask=memory_mask)
34            return self.fc_out(output)
35
36        @staticmethod
37        def generate_square_subsequent_mask(sz):

```



```

35     """Tạo causal mask cho decoder."""
36     mask = torch.triu(torch.ones(sz, sz), diagonal=1).bool()
37     return mask # True = vị trí bị mask

```

Listing 7: Transformer hoàn chỉnh

Ví dụ: Sử dụng mô hình

```

# Khởi tạo
model = TransformerModel(
    src_vocab=10000, tgt_vocab=8000,
    d_model=256, n_heads=8, n_layers=4, d_ff=1024
)
# Dữ liệu giả
src = torch.randint(0, 10000, (32, 20)) # batch=32, len=20
tgt = torch.randint(0, 8000, (32, 15)) # batch=32, len=15
tgt_mask = model.generate_square_subsequent_mask(15)
# Forward pass
out = model(src, tgt, tgt_mask=tgt_mask)
print(out.shape) # torch.Size([32, 15, 8000])

```

8 Các mô hình Transformer huấn luyện sẵn

8.1 Tổng quan các mô hình phổ biến

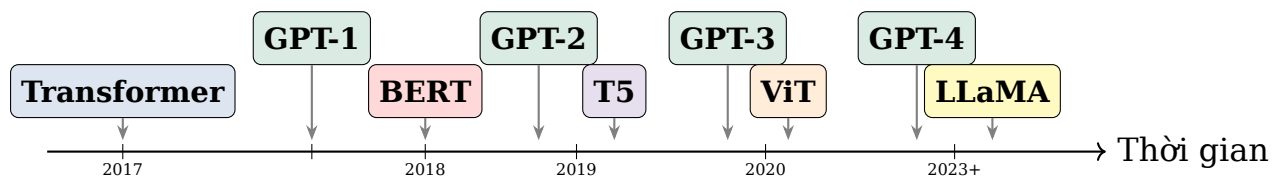


Figure 11: Dòng thời gian các mô hình Transformer quan trọng.

8.1.1 BERT (Bidirectional Encoder Representations from Transformers)

- **Kiến trúc:** Chỉ dùng Encoder (encoder-only).
- **Tiền huấn luyện:** Masked Language Modeling (MLM) — che 15% token, yêu cầu mô hình dự đoán token bị che; và Next Sentence Prediction (NSP).
- **Ưu điểm:** Đọc ngữ cảnh *hai chiều* (bidirectional), rất mạnh cho các tác vụ hiểu ngôn ngữ (understanding).
- **Kích thước:** BERT-base: 12 lớp, 768 chiều, 110M tham số.

8.1.2 GPT (Generative Pre-trained Transformer)

- **Kiến trúc:** Chỉ dùng Decoder (decoder-only) với causal masking.

- **Tiền huấn luyện:** Causal Language Modeling — dự đoán token tiếp theo.
- **Ưu điểm:** Rất mạnh cho sinh văn bản (generation), có khả năng few-shot learning.
- **GPT-3:** 175 tỷ tham số, 96 lớp, $d_{model} = 12288$.

Table 3: So sánh BERT vs. GPT

Tiêu chí	BERT	GPT
Kiến trúc	Encoder-only	Decoder-only
Hướng ngữ cảnh	Hai chiều	Một chiều (trái → phải)
Mục tiêu huấn luyện	MLM + NSP	Causal LM
Thế mạnh	Hiểu ngôn ngữ	Sinh ngôn ngữ
Fine-tuning	Thêm head cho từng tác vụ	Prompt-based / fine-tune

8.2 Sử dụng thư viện Hugging Face Transformers

```

1 from transformers import pipeline
2
3 # Tạo pipeline phân loại cảm xúc
4 classifier = pipeline("sentiment-analysis",
5                       model="nlpTown/bert-base-multilingual-
6                           uncased-sentiment")
7
8 # Dự đoán
9 results = classifier([
10     "Khách sạn này rất tuyệt vời!",
11     "Dịch vụ tệ quá, tôi rất thất vọng."
12 ])
13 for r in results:
14     print(f"Label: {r['label']}, Score: {r['score']:.4f}")

```

Listing 8: Dùng Hugging Face để phân loại cảm xúc

```

1 from transformers import pipeline
2
3 unmasker = pipeline("fill-mask", model="bert-base-uncased")
4 results = unmasker("The capital of France is [MASK].")
5 for r in results[:3]:
6     print(f"{r['token_str']:>10} | score = {r['score']:.4f}")
7 # Output:
8 #     paris | score = 0.9836
9 #     france | score = 0.0054
10 #     lyon | score = 0.0018

```

Listing 9: Dùng BERT cho Masked Language Modeling

```

1 from transformers import pipeline
2

```

```
3 generator = pipeline("text-generation", model="gpt2")
4 output = generator(
5     "Deep learning has transformed",
6     max_length=50,
7     num_return_sequences=1,
8     temperature=0.7
9 )
10 print(output[0]['generated_text'])
```

Listing 10: Dùng GPT-2 để sinh văn bản

9 Fine-tuning mô hình Transformer huấn luyện sẵn

9.1 Khái niệm và kỹ thuật

Định nghĩa: Fine-tuning

Fine-tuning là quá trình lấy một mô hình đã được tiền huấn luyện (pre-trained) trên tập dữ liệu lớn, sau đó tiếp tục huấn luyện trên tập dữ liệu nhỏ hơn cho một tác vụ cụ thể.

Các chiến lược fine-tuning phổ biến:

1. **Full fine-tuning**: Cập nhật tất cả tham số. Hiệu quả cao nhưng tốn tài nguyên.
2. **Feature extraction (đóng băng)**: Giữ nguyên trọng số backbone, chỉ huấn luyện classification head.
3. **Gradual unfreezing**: Mở dần các lớp từ trên xuống dưới.
4. **LoRA (Low-Rank Adaptation)**: Thêm ma trận low-rank vào các lớp attention, chỉ huấn luyện phần thêm vào — rất tiết kiệm bộ nhớ.

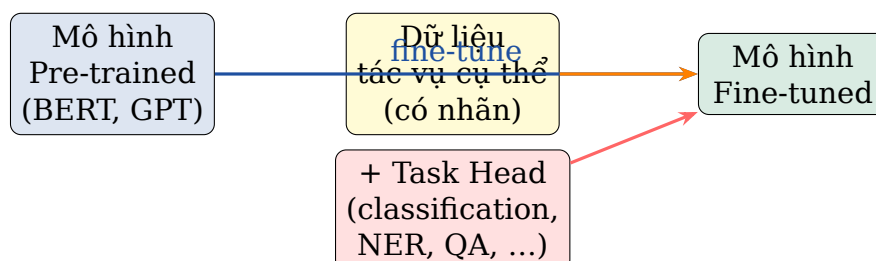


Figure 12: Pipeline fine-tuning: mô hình pre-trained + dữ liệu tác vụ + task head → mô hình fine-tuned.

9.2 Ví dụ: Phân loại văn bản với Hugging Face

```
1 from transformers import (  
2     AutoTokenizer, AutoModelForSequenceClassification,  
3     TrainingArguments, Trainer  
4 )  
5 from datasets import load_dataset  
6 import numpy as np  
7 from sklearn.metrics import accuracy_score  
8  
9 # 1. Tải dữ liệu  
10 dataset = load_dataset("imdb")  
11  
12 # 2. Tokenize  
13 tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")  
14  
15 def tokenize_fn(examples):  
16     return tokenizer(  
17         examples["text"], padding="max_length",  
18         truncation=True, max_length=256  
19     )  
20  
21 tokenized = dataset.map(tokenize_fn, batched=True)  
22  
23 # 3. Tải mô hình pre-trained + classification head  
24 model = AutoModelForSequenceClassification.from_pretrained(  
25     "bert-base-uncased", num_labels=2  
26 )  
27  
28 # 4. Cấu hình huấn luyện  
29 training_args = TrainingArguments(  
30     output_dir="./results",  
31     num_train_epochs=3,  
32     per_device_train_batch_size=16,  
33     per_device_eval_batch_size=64,  
34     learning_rate=2e-5,  
35     weight_decay=0.01,  
36     eval_strategy="epoch",  
37     save_strategy="epoch",  
38     load_best_model_at_end=True,  
39 )  
40  
41 # 5. Định nghĩa metric  
42 def compute_metrics(eval_pred):  
43     logits, labels = eval_pred  
44     preds = np.argmax(logits, axis=-1)  
45     return {"accuracy": accuracy_score(labels, preds)}  
46  
47 # 6. Tạo Trainer và huấn luyện  
48 trainer = Trainer(  
49     model=model,
```

```

50     args=training_args,
51     train_dataset=tokenized["train"].select(range(5000)),
52     eval_dataset=tokenized["test"].select(range(1000)),
53     compute_metrics=compute_metrics,
54 )
55
56 trainer.train()
57
58 # 7. Danh gia
59 results = trainer.evaluate()
60 print(f"Accuracy: {results['eval_accuracy']:.4f}")

```

Listing 11: Fine-tuning BERT cho phân loại văn bản

Mẹo: Mẹo fine-tuning hiệu quả

- **Learning rate nhỏ:** 2×10^{-5} đến 5×10^{-5} cho BERT. Quá lớn sẽ phá hủy kiến thức pre-trained (catastrophic forgetting).
- **Warmup:** Tăng dần learning rate trong vài bước đầu, sau đó giảm dần.
- **Batch size:** 16-32. Nếu GPU không đủ bộ nhớ, dùng gradient accumulation.
- **Epochs:** 2-5 epochs thường đủ. Quá nhiều dễ overfit.
- **LoRA:** Với GPU hạn chế, dùng thư viện peft để chỉ fine-tune < 1% tham số.

9.3 Ví dụ nhanh: Fine-tuning với LoRA

```

1 from peft import LoraConfig, get_peft_model, TaskType
2
3 # Cấu hình LoRA
4 lora_config = LoraConfig(
5     task_type=TaskType.SEQ_CLS,
6     r=8, # Rank của ma trận low-rank
7     lora_alpha=32, # Hệ số scale
8     lora_dropout=0.1,
9     target_modules=["query", "value"], # Chỉ áp dụng cho Q, V
10 )
11
12 # Bọc mô hình với LoRA
13 model = AutoModelForSequenceClassification.from_pretrained(
14     "bert-base-uncased", num_labels=2
15 )
16 peft_model = get_peft_model(model, lora_config)
17
18 # Kiểm tra số tham số trainable
19 peft_model.print_trainable_parameters()
20 # Output: trainable params: 294,914 || all params: 109,776,386
21 #           || trainable%: 0.2686

```

Listing 12: Fine-tuning với LoRA (tiết kiệm tài nguyên)

Bảng tóm tắt các khái niệm chính

Khái niệm	Công thức / Mô tả ngắn
Attention weights	$\alpha_{t,i} = \text{softmax}(e_{t,i})$
Context vector	$c_t = \sum_i \alpha_{t,i} h_i$
Scaled Dot-Product	$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
Multi-Head	$\text{MH} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$
Positional Encoding	$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$
FFN	$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$
Add & Norm	$\text{LN}(x + \text{Sublayer}(x))$
BERT	Encoder-only, MLM + NSP, hai chiều
GPT	Decoder-only, Causal LM, một chiều
ViT	Chia ảnh thành patch $\rightarrow$ Transformer Encoder
Fine-tuning	Pre-trained model + task-specific data + head
LoRA	Thêm ma trận low-rank ($r \ll d$) vào attention layers

Tài liệu tham khảo

- [1] Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- [2] Bahdanau, D. et al. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015.
- [3] Luong, M.T. et al. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP.
- [4] Devlin, J. et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019.
- [5] Radford, A. et al. (2018, 2019). *Improving Language Understanding by Generative Pre-Training (GPT); Language Models are Unsupervised Multitask Learners (GPT-2)*.
- [6] Brown, T. et al. (2020). *Language Models are Few-Shot Learners (GPT-3)*. NeurIPS.
- [7] Dosovitskiy, A. et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT)*. ICLR 2021.
- [8] Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- [9] Wolf, T. et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Hugging Face.